

実験ログ: 煙と炎 (Pyro・ボリューム)

2026-09-24 | 収録 9 件

Houdini 研究部で、hython で実際に測った実験の記録。数字はどれも測った値。

収録している実験

- Pyro 煙 — dissipation の正体を式で突き止める
- 煙はなぜ動かなかったのか — 温度を入れると立ち上る
- 炎を出す — shredding が効かなかった理由
- 動く煙で撮り直す — 018の結論は変わらなかった
- 炎に色を付ける — 黒体放射の式を数字で追う
- キャッシュを道具に組み込む — 古いファイルを読む事故を防ぐ
- 升目を半分にしても、数は8倍ではなく約4倍 — VDB は表面の近くだけ持っている
- Pyro で焚き火の炎（高さ 1.2 m）を作るなら Flame Lifespan 0.25 ・ Buoyancy 0.25 ・ Cooling Rate 1 — 揺らすのは Turbulence で、Use Control Field を切る
- 焚き火の Pyro を速く回すなら Voxel Size を 0.02 → 0.04 に — 5.1 倍速く、炎の高さの差は 3%。速度の升だけ粗くすると Substeps 2 は、かえって遅い

Pyro 煙 — dissipation の正体を式で突き止める

2026-09-24 | Houdini 21.0.700 | /obj/pyro_test | ノード 6 / 接続 5

何を作ったか

剛体は体積、布は辺の長さが保たれた。煙はボリウム（空間を格子で区切って各マスに値を持たせたもの）なので、測るのは密度の合計になる。

当初は「煙は消えていくので密度は減るはず」と考えたが、実際には増えた。発生源が毎フレーム供給し続けるためで、減衰と供給が釣り合う。そこで「供給と減衰の釣り合い」を式に立てて実測と突き合わせたところ、小数点以下4桁まで一致した。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.5 rady = 0.5 radz = 0.5 ty = 0.6 type = 1
source	pyrosource	attribute1 = 0 attributes = 1 mode = 2
rasterize	volumerasterizeattributes	attributes = density folder1 = 1 folder3 = 1 voxelsize = 0.08
solve_d0_0	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation = 0.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 …ほか 102 件

ノード	タイプ	デフォルトから変更したパラメータ
solve_d0_1	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 disturbance_controlramp2value = 1.0 …ほか 100 件
solve_d0_3	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation = 0.3 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 …ほか 101 件

密度の合計の変化（最初のフレームを1とした比）

半径0.5の球から煙を出し続けた。dissipation は煙が消える速さのパラメータで、既定は 0.1。

フレーム	散逸なし (0.0)	既定 (0.1)	強め (0.3)
1	1.00000	1.00000	1.00000
5	5.00000	4.09489	2.77298
10	10.00000	6.51284	3.23902
20	20.00000	8.78371	3.33052
30	30.00000	9.57551	3.33310
46	46.00000	9.92084	3.33318

立てた式と実測の突き合わせ

供給が毎フレーム一定で、毎フレーム $(1 - d)$ 倍に減るとすれば、 t フレーム後の合計は $(1 - (1-d)^t) / d$ 倍になるはず。この式と実測を比べた。

条件	フレーム	実測の比	式の予測	差
散逸なし	10	10.00000	10.00000	0.000000
散逸なし	46	46.00000	46.00000	0.000000
既定 0.1	5	4.09489	4.09510	-0.000210
既定 0.1	20	8.78371	8.78423	-0.000523
既定 0.1	46	9.92084	9.92145	-0.000608

条件	フレーム	実測の比	式の予測	差
強め 0.3	5	2.77298	2.77310	-0.000120
強め 0.3	46	3.33318	3.33333	-0.000153

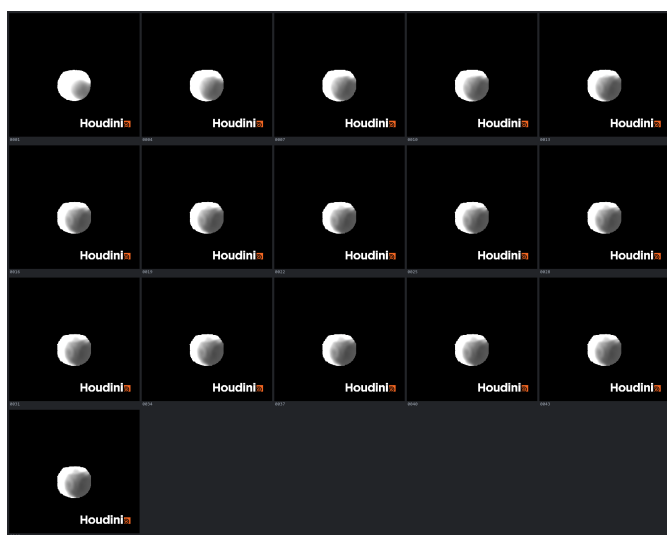
行き着く先（釣り合いの値）

供給と減衰が釣り合うと、密度はある値で止まる。式の上では $1 / \text{dissipation}$ になるはず。

条件	最後の実測	$1 / \text{dissipation}$	差
既定 0.1	9.92084	10.00000	0.079
強め 0.3	3.33318	3.33333	0.00015

見た目（16コマ）

数値ははっきりしているが、画はあまり情報を持っていない。理由は下の「わかったこと」に書いた。



16コマ。ハードウェアレンダラーはポリウムを塊としてしか描けない

わかったこと

- ・当初の予想は外れた。「煙は消えていくので密度は減る」と考えていたが、実際には増えた。発生源が毎フレーム煙を供給し続けるため、減る量より足される量が多い間は増え続ける。
- ・散逸なし（ $\text{dissipation} = 0$ ）のとき、密度の合計はフレーム番号とぴったり同じ比になった。5フレーム目で5.000000倍、46フレーム目で46.000000倍。差は完全にゼロ。毎フレーム同じ量が足され、一切減らないので当然だが、これがきれいにすることでシミュレーションが余計なことをしていないと確認できる。
- ・ dissipation は「毎フレーム $(1 - d)$ 倍に減らす」という意味だと確定した。この仮定から立てた式 $(1 - (1-d)^t) / d$ と実測を比べると、全フレームで差が 0.0006 以下（最大でも 0.006%）。
- ・行き着く先も式のとおり。供給と減衰が釣り合う値は $1 / \text{dissipation}$ で、0.3 のとき理論値 3.33333 に対し実測 3.33318。0.1 のときは 10 に対し 9.92084 で、まだ完全には収束していないが確実に近づいている。
- ・この形の検証は応用が効く。パラメータの意味が分からないときは、仮定を式にして実測と突き合わせればよい。説明を読むより確実に、桁が合っているかまで分かる。実験015で「かたさ = 値 × 10の指数乗」を突き止めたのと同じやり方。

- ・ボリウムの構成でつまづいた点。pyrosource は既定で P と pscale しか作らず、density 属性を明示的に追加しないと次のラスライズが空を返す。そして紛らわしいことに、pyrosource の attributes は「作る属性の個数」（数値）、volumerasterizeattributes の attributes は「属性名を並べた文字列」で、同じ名前なのに意味が違う。

- ・ハードウェアレンダラー（OpenGL ROP）はボリウムの描画に向いていない。煙が塊のようにしか描かれず、濃淡も立ち上りも分からない。これまでの実験ではこのレンダラーで十分だったが、ボリウムを扱うならKarmaのような本式のレンダラーが要る。

- ・計算は速く、1フレームあたり 0.04~0.2秒。ボリウムでもこの規模なら実験のテンポは保てる。

次にやること

- ・Karma でボリウムをレンダリングし、煙らしい見た目を出す。レンダリング時間も測る
- ・buoyancylift（浮力）を変えて、立ち上る速さがどう変わるかを測る
- ・FLIP（液体）へ。液体なら体積が保存されるはずなので、また別の検証軸になる

煙はなぜ動かなかったのか — 温度を入れると立ち上る

2026-09-24 | Houdini 21.0.700 | /obj/smoke | ノード 4 / 接続 3

何を作ったか

実験018で「いまの煙は一様な球で、レンダラーの差を見せる題材として弱い」と書いた。形のある煙を作ろうとして測ったら、もっと手前に問題があった。

煙がまったく動いていなかった。中身のあるボクセルの数が全フレームで504のまま、ばらつきも1.4927のまま。合計だけが増えていく。これは「同じ形が濃くなっていくだけ」を意味する。昇りも広がりもしていない。実験016の煙も、実は同じだった。

浮力は温度で決まる。density だけを供給して temperature を供給していないので、昇る理由がなかった。temperature を足したら、格子が [16,32,16] から [48,144,48] に伸び、中身のあるボクセルが28倍になった。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.22 radx = 0.22 radz = 0.22 rows = 24 ty = -0.9 type = 2
src	pyrosource	attribute1 = 0 attribute2 = 1 attributes = 2
rasterize	volumerasterizeattributes	attributes = density temperature folder1 = 1 folder3 = 1 voxelsize = 0.05

ノード	タイプ	デフォルトから変更したパラメータ
solve	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance = 2.0 disturbance_controlramp2pos = 1.0 …ほか 101 件

density だけを供給したとき

「中身」は密度が0より大きいボクセルの数。「ばらつき」は変動係数（標準偏差÷平均）で、全体を何倍しても変わらない量。形が変われば動くが、濃さが変わるだけなら動かない。

フレーム	密度の合計	中身のあるボクセル	重心の高さ	ばらつき	格子
5	17,379.98	504	0.4677	1.4927	16×32×16
15	33,702.64	504	0.4677	1.4927	16×32×16
25	39,394.00	504	0.4677	1.4927	16×32×16
40	41,813.50	504	0.4677	1.4927	16×32×16

temperature も供給したとき

同じ組み方で、供給する属性を1つ足しただけ。

フレーム	密度の合計	中身のあるボクセル	重心の高さ	ばらつき	格子
5	18,356.35	1,072	0.5718	1.6673	48×32×48
15	33,901.98	5,851	0.5109	1.9247	48×64×48
25	41,407.58	14,006	0.5148	2.7519	48×96×48
40	48,289.94	30,105	0.5217	3.7187	48×144×48

かき乱す設定は効いたか（40フレーム目）

disturbance と shredding を、温度ありの状態に足して比べた。

条件	密度の合計	中身	ばらつき	温度ありとの差
温度あり（基準）	48,289.94	30,105	3.7187	—
disturbance 2.0	52,247.42	30,350	3.5663	合計 +8.2%
shredding 1.0	48,289.94	30,105	3.7187	完全に同じ
shredding 20.0	48,289.94	30,105	3.7187	完全に同じ

わかったこと

- ・煙は動いていなかった。density だけを供給した場合、中身のあるボクセルは全フレームで 504個、ばらつきは 1.4927、重心の高さは 0.4677。どれも1桁も動かない。合計だけが増えていく。つまり同じ形が濃くなっていくだけで、昇りも広がりもしていない。

- ・格子の大きさも 16×32×16 のまま変わらなかった。煙が広がれば計算領域も広がるはずなので、これも「動いていない」ことの裏付けになる。

- ・原因は温度を供給していなかったこと。浮力は温度差で生まれる。密度だけ渡しても、昇る理由がない。pyrosource で属性を2つにして temperature を足し、volumerasterizeattributes にも density temperature と書いたら動き出した。

- ・ temperature を足した結果、格子は [48,32,48] から [48,144,48] へ縦に4.5倍伸び、中身のあるボクセルは 1,072 から 30,105 へ28倍になった。ばらつきも 1.6673 から 3.7187 へ 2.2倍で、形ができています。

- ・実験016と018の煙も同じ状態だった。016で測った散逸の式（毎フレーム (1-d) 倍）は密度の増減だけの話なので、結論は変わらない。018のノイズの式もレンダラー側の話なので変わらない。ただしどちらも「動かない煙」を撮っていたので、絵が地味だったのはこれが理由。

- ・かき乱す設定も試した。disturbance は効いた。合計が 8.2% 増え、ばらつきは少し下がる (3.7187 → 3.5663)。細かくばらけさせる設定なので、ばらつきが下がるのは筋が通る。ただし合計が動いたので「動かすだけ」ではない。私の最初の仮説は外れた。

- ・shredding はまったく効かなかった。1.0 でも 20.0 でも、数値が小数点以下まで基準と完全に一致する。付属ヘルプには「温度の勾配にもとづいて速度場を押し引きし、炎に特有の筋や裂け目を作る」とある。炎（燃焼）を伴わない煙だけの設定では出番がない、というのが今のところの読み。ここは確かめきれていないので、炎を出す設定で改めて試す。

- ・ヘルプには「温度の拡散が小さいほど効果が出やすい」とも書いてあったが、tempdiffusion は既定で 0.0、つまりすでに最小だった。この線では説明がつかない。

次にやること

- ・炎（燃焼）を出す設定にして、shredding が効くかどうかを確かめる
- ・この煙で Karma を撮り直す。実験018は動かない煙で比べていた
- ・022 として、液体に表面を張る (particlefluidsurface)

炎を出す — shredding が効かなかった理由

2026-09-24 | Houdini 21.0.700 | /obj/burn | ノード 4 / 接続 3

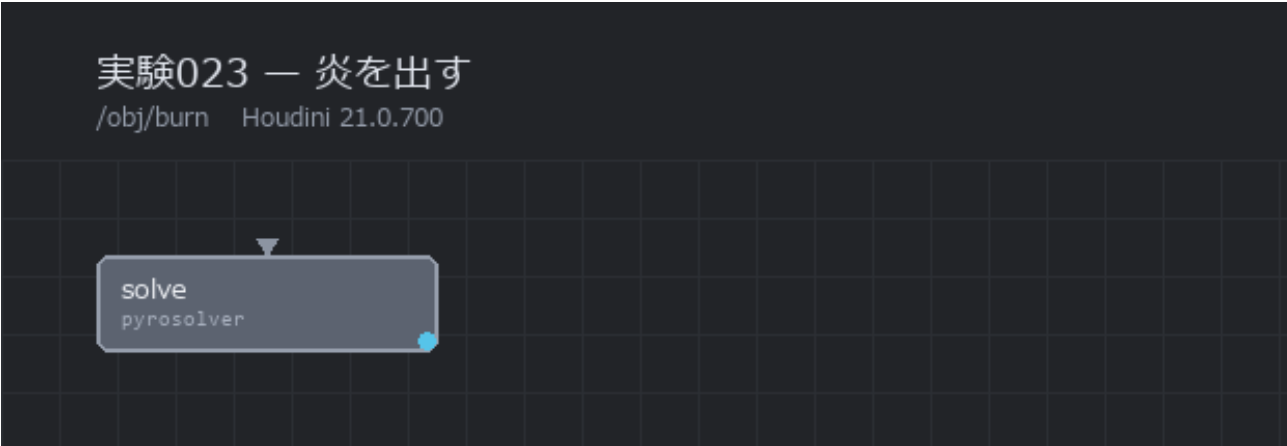
何を作ったか

実験021で shredding を 1.0 でも 20.0 でも試したが、数値が小数点以下まで完全に一致した。まったく効いていない。

付属ヘルプにはこうある。「温度の勾配にもとづいて速度場を押し引きし、炎に特有の筋・裂け目・舌のような形を作る」。021では「炎を伴わない煙だけの設定では出番がないのだろう」と書いたが、そこで止めていた。確かめていない見立てを残したままにしたくないので、今回それを確かめた。

燃烧を入れると shredding は効いた。炎の合計が40%減り、形のばらつきが9%増え、立ち上る高さも縮んだ。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.22 rady = 0.22 radz = 0.22 rows = 24 ty = -0.9 type = 2
src	pyrosource	attribute1 = 2 attribute2 = 1 attributes = 2 name1 = burn name2 = temperature
rasterize	volumerasterizeattributes	attributes = burn temperature folder1 = 1 folder3 = 1 voxelsize = 0.05

ノード	タイプ	デフォルトから変更したパラメータ
solve	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 disturbance_controlramp2value = 1.0 …ほか 103 件

煙だけのとき（021の再確認）

供給するのは density と temperature。shredding を 0 と 5.0 で比べた。

条件	F40の合計	中身のマス	ばらつき	格子
shredding なし	48,289.94	30,105	3.7187	48×144×48
shredding 5.0	48,289.94	30,105	3.7187	48×144×48
差	0	0	0	なし

燃焼を入れたとき

pyrosource の初期化プリセット sourceburn を使うと、作られる属性が burn と temperature になる。density は作られないので、どの条件にもある temperature を主な指標にした。

条件	F40の温度の合計	中身のマス	ばらつき	格子
shredding なし	4,724.59	38,637	1.8278	48×144×48
shredding 5.0	3,673.59	41,569	1.9914	48×128×48
差	−22.2%	+7.6%	+9.0%	縦が16マス短い

炎そのものの量

flame の場（燃えている部分）だけを見たもの。

条件	F25の合計	F25のマス	F40の合計	F40のマス
shredding なし	3,001.8	7,374	3,727.4	10,621
shredding 5.0	2,424.4	7,314	2,231.5	7,906
差	−19.2%	−0.8%	−40.1%	−25.6%

フレームごとの推移（燃焼あり）

温度の合計と、形のばらつき。

フレーム	なしの合計	ありの合計	なしのばらつき	ありのばらつき
5	765.31	760.67	0.8442	0.8427
15	2,106.41	1,994.95	1.0784	1.1381

フレーム	なしの合計	ありの合計	なしのばらつき	ありのばらつき
25	3,281.97	2,853.95	1.4196	1.5271
40	4,724.59	3,673.59	1.8278	1.9914

わかったこと

- ・021の見立ては当たっていた。shredding は炎（燃焼）があるときだけ効く。煙だけの設定では、値を 0 から 5.0 に変えても数値が1桁も動かない。燃焼を入れると、同じ変更ですべての数字が動く。
- ・効き方にも筋が通っている。炎の合計が40.1%減り、燃えているマスが25.6%減る一方で、温度の広がり（中身のマス）は7.6%増え、ばらつきは9.0%増えた。つまり炎がちぎれて薄く広がり、そのぶん早く燃え尽きる。ヘルプの「筋・裂け目・舌のような形」という説明と合う。
- ・立ち上る高さも縮む。計算領域が 48×144×48 から 48×128×48 になった。縦に16マス短い。ちぎれた炎は上昇の勢いを失いやすい。
- ・違いはフレームが進むほど大きくなる。F5では合計の差が0.6%しかないが、F40では22.2%になる。短いフレームだけ見て「効かない」と判断すると間違える。021で気づけなかった一因でもある。
- ・燃焼の組み方。pyrosource の初期化に sourceburn というプリセットがある。これを押すと、作られる属性が density と temperature ではなく burn と temperature に変わる。burn が「燃える元」で、ソルバがこれを炎に変える。
- ・ソルバ側では addflamefield を有効にすると flame の場ができる。煙も出したい場合は doflamedensity を有効にするが、量の既定が 0.0001 でほとんど出ない。1.0 にして初めて見えるようになった。
- ・この実験で density の合計が 0 のままだったのは、燃焼のプリセットが density を作らないため。指標を temperature に変えて測り直した。「測ろうとしたものが存在しない」と気づけたのは、フィールドの一覧を毎回出していたから。
- ・021で「確かめきれしていない」と書いて残した見立てを、2実験あとで潰せた。見立てを断定せずに残しておく、あとで回収できる。

次にやること

- ・024 として、動く煙で Karma を撮り直す。実験018は動かない煙で比べていた
- ・炎に色を付ける。いまは白飛びしていて炎に見えない
- ・025 として、シミュレーションのキャッシュ（file cache）に対応する

動く煙で撮り直す — 018の結論は変わらなかった

2026-09-24 | Houdini 21.0.700 | /obj/smoke | ノード 4 / 接続 3

何を作ったか

実験018で2つのレンダラーを比べたとき、題材にしていた煙は動いていなかった。それが分かったのは実験021。一様な球が濃くなるだけの絵で比べていたことになる。

題材が悪いと結論も疑わしい。021で立ち上るようにした煙に入れ替えて、同じ指標で測り直した。

結果、数字はほとんど変わらなかった。階調は 235→233 と 200→200、最も暗い画素は 15→17 と 50→49。018の結論はそのまま成り立つ。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.22 rad y = 0.22 radz = 0.22 rows = 24 ty = -0.9 type = 2
src	pyrosource	attribute1 = 0 attribute2 = 1 attributes = 2
rasterize	volumerasterizeattributes	attributes = density temperature folder1 = 1 folder3 = 1 voxelsize = 0.05

ノード	タイプ	デフォルトから変更したパラメータ
solve	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 disturbance_controlramp2value = 1.0 …ほか 99 件

動く煙を2つのレンダラーで（同じカメラ・同じ露出）

実験021で温度を供給して立ち上るようにした煙。フレーム40、460×760、Karma は256サンプル。

レンダラー	時間	煙の画素	階調	平均の明るさ	ばらつき	最も暗い
OpenGL ROP	0.73 秒	33,003	233 段	129.9	72.99	17
Karma 256サンプル	4.30 秒	25,726	200 段	141.5	49.63	49

実験018（動かない煙）との比較

題材を入れ替えても、どの指標もほとんど動かない。

指標	018 OpenGL	018 Karma	024 OpenGL	024 Karma
階調	235	200	233	200
ばらつき	86.16	52.83	72.99	49.63
最も暗い画素	15	50	17	49

煙の中心を横切る線の明るさ

255 が背景。数字が小さいほど濃い。

レンダラー	縁の外	中心	縁の外
OpenGL ROP	254	42	253
Karma	255	93	255

わかったこと

- ・018の結論は変わらなかった。題材を「動かない球」から「立ち上る煙」に入れ替えても、階調は 235→233 と 200→200、最も暗い画素は 15→17 と 50→49。どれも誤差の範囲でしか動かない。
- ・これは2つのレンダラーの違いが、題材の複雑さに左右されないことを意味する。光が中を通るかどうかという違いは、形が単純でも複雑でも同じように出る。
- ・疑って測り直したこと自体には意味がある。018の結論は「悪い題材で出した結論かもしれない」という弱点を抱えていた。測り直したことでその弱点が消えた。結果が変わらなくても、確かめる価値はある。

- ・見た目の差ははっきりしている。OpenGL

は煙の頭も柱も一様に暗く、どちらから光が当たっているか分からない。Karma

は頭の内側に陰影が出て、丸みが分かる。数字では「最も暗い画素」がこの違いを捉えている（17 対 49）。OpenGL は濃いところをほぼ黒まで落とすが、Karma は光が抜けてくるぶん薄く留まる。

- ・速さの差も018と同じ傾向。0.73秒 対 4.30秒で約5.9倍（018では5.1倍）。確認用は OpenGL、見せる絵は Karma、という使い分けはそのままよい。

- ・今回の煙は実験021の組み方をそのまま使った。temperature を供給しないと煙は動かない。この1行がないと、また「動かない球」を撮ることになる。

次にやること

- ・炎に色を付ける。いまは白黒でしか描けていない
- ・025 として、シミュレーションのキャッシュ（file cache）に対応する
- ・実験030に達したら、001～030の振り返り点検を行う

炎に色を付ける — 黒体放射の式を数字で追う

2026-09-24 | Houdini 21.0.700 | /obj/fire | ノード 7 / 接続 7

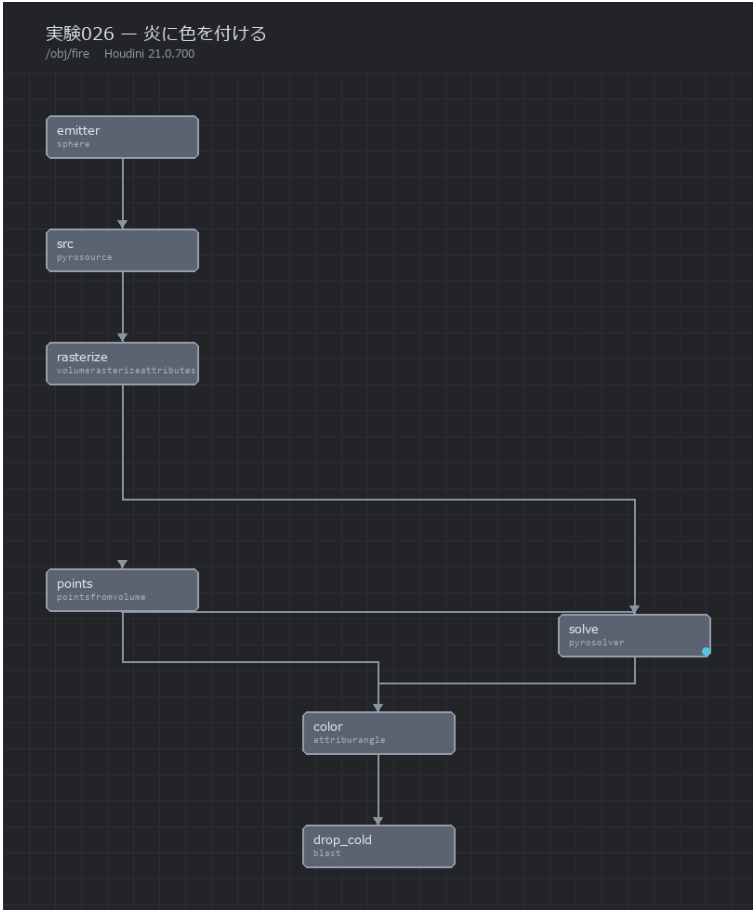
何を作ったか

実験023で炎を出したが、白黒でしか描けていなかった。炎は温度で色が変わる。低いと赤、高くなるにつれて橙・黄・白へ。これは黒体放射という物理の話で、Houdini の VEX には `blackbody()` という関数がそのまま入っている。

「色が付いた」で終わらせず、温度ごとに返る色を数字で取り出した。赤成分と青成分の比は 1000K で 17.39、8000K で 0.7381。23.6分の1まで下がり、温度とともにきれいに単調に減る。

見た目にするまでに2回つまずいた。返る値が 6500倍ほどの大きさだったこと、そして 1 で切ると赤が潰れて全部黄色になることである。

ノードの接続



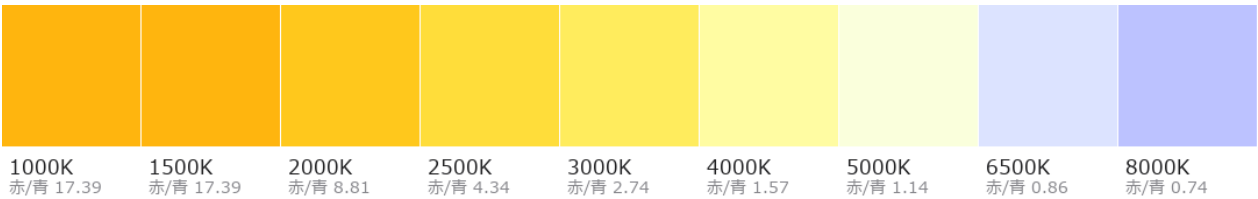
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.22 rady = 0.22 radz = 0.22 rows = 24 ty = -0.9 type = 2

ノード	タイプ	デフォルトから変更したパラメータ
src	pyrosource	attribute1 = 2 attribute2 = 1 attributes = 2 name1 = burn name2 = temperature
rasterize	volumerasterizeattributes	attributes = burn temperature folder1 = 1 folder3 = 1 voxelsize = 0.05
solve	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 disturbance_controlramp2value = 1.0 …ほか 101 件
points	pointsfromvolume	particlesep = 0.080000000000000002
color	attribwrangle	snippet = // 2本目の入力（ボリウム）から、この点の位置の温度を読む float t = volumesample(1, "temperature", @P); f@temp = t; // 温度の単位をケルビンに直してから色にする vec...
drop_cold	blast	group = @temp<0.02 grouptype = 3

blackbody() が温度ごとに返す色

2つ目の引数は「白として扱う温度」で 6500 を指定した。値は 6500 で割って 0～1 に収めてある。赤と青の比を見ると、色の変化が数字で追える。



左が 1000K、右が 8000K。各マスの下は 赤/青 の比。

温度(K)	赤	緑	青	赤/青
1000	1.4015	1.0000	0.0806	17.3872
1500	1.4015	1.0000	0.0806	17.3872
2000	1.2750	1.0000	0.1448	8.8063
2500	1.1517	1.0000	0.2655	4.3383
3000	1.0802	1.0000	0.3940	2.7415
4000	1.0101	1.0000	0.6443	1.5676
5000	0.9812	1.0000	0.8630	1.1370

温度(K)	赤	緑	青	赤/青
6500	0.9686	1.0000	1.1211	0.8640
8000	0.9686	1.0000	1.3122	0.7381

炎の温度を色にする

ボリウムの中に点をばらまき、点の位置の温度を読んで色にした。点なら確認用の描画（OpenGL）でも色が出る。



実験023の炎。温度は持っているが色になっていない。 温度を色にしたもの。根元が橙、上が黄。

調べたこと	結果
ばらまいた点	4,779 個
温度の範囲	0.3227 ~ 0.9929
ケルビンに直すと (×3000)	968K ~ 2,979K
その範囲の赤/青	約 17.4 ~ 2.8

わかったこと

- ・ 赤と青の比は、温度とともにきれいに単調に下がった。1000K で 17.39、8000K で 0.7381。23.6分の1になる。9点すべてで下がる向きが崩れない。式は思ったとおりに効いている。
- ・ 1000K と 1500K が完全に同じ値だった。どちらも (1.4015, 1.0000, 0.0806)。低いほうで頭打ちになっている。炎の一番冷たいところを描き分けたいなら、この下限より上に収まるよう温度の換算を調整する必要がある。
- ・ 引数は2つ。blackbody(温度) と書くとエラーになる。正しくは blackbody(温度, 白として扱う温度)。エラーメッセージが候補を出してくれるので気づける。
- ・ 返る値は 0~1 ではない。6500 を指定すると、緑成分がちょうど 6500 になる大きさに返ってくる。そのまま色に入れると全部真っ白になる。6500 で割って初めて見える値になった。
- ・ 1 で切ると色味が消える。1000K の (1.4015, 1.0000, 0.0806) を 0~1 に丸めると (1.0, 1.0, 0.08) になり、赤と緑が同じ値 = 黄色になってしまう。実際、最初の画は全部黄色だった。

- ・直し方は「一番大きい成分で割る」。(1.4015, 1.0, 0.0806) ÷ 1.4015 = (1.0, 0.713, 0.058)

で、橙赤のまま明るさだけが揃う。丸めるか割るかで、色が別物になる。

- ・ボリュームそのものには色を付けず、pointsfromvolume で中に点をばらまいて点に色を付けた。こうすると確認用の速い描画でも色が見える。見せる絵にするときは Karma 側でボリュームに色を付ける。

- ・点から温度を読むのは volumesample(1, "temperature", @P)。2本目の入力につないだボリュームを、その点の位置で読む。

次にやること

- ・ Karma でボリュームに直接色を付ける。いまは点に色を付けて代用している
- ・ 027 として、hou_tools にキャッシュを組み込む
- ・ 実験030に達したら、001～030の振り返り点検を行う

キャッシュを道具に組み込む ー 古いファイルを読む事故を防ぐ

2026-09-24 | Houdini 21.0.700 | /obj/smoke | ノード 5 / 接続 4

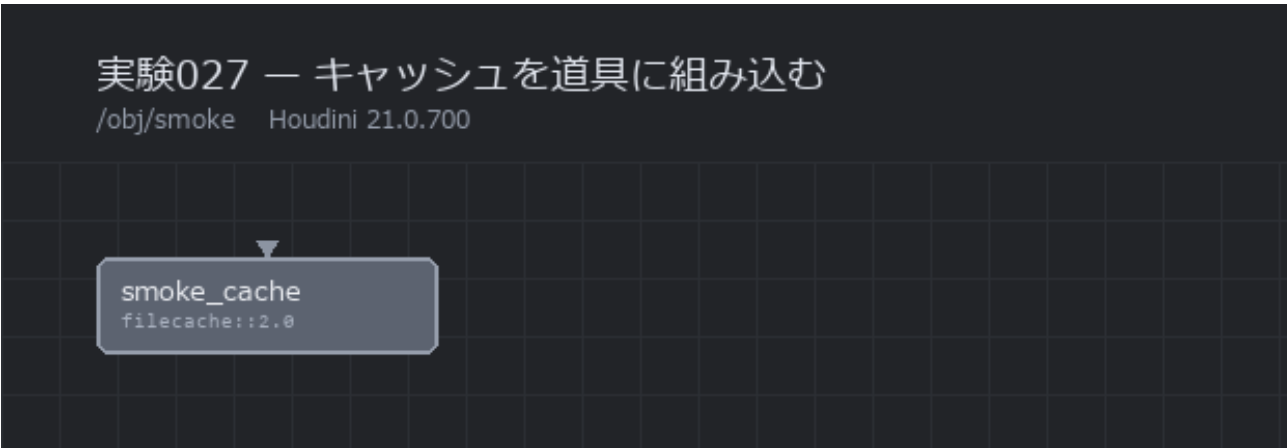
何を作ったか

実験025で filecache の挙動は測った。読み込みは速く、値は完全に一致する。ただし手で組むと毎回同じ手間がかかるうえ、もっと危ない落とし穴がある。

上流を変えたのに、古いキャッシュを読んでしまうことである。このとき画も数字も出てくる。エラーも出ない。ただ、それは前の設定の結果である。測った値が静かに嘘になる。このプロジェクトで一番避けたい事故がこれになる。

そこで hou_tools.cache_sim()
に指紋を持たせた。上流のノードの種類と「既定から動かしたパラメータ」をまとめて SHA-1 を取り、キャッシュの隣に置く。一致したときだけ読み、変わっていれば黙って計算し直す。仕組みが効いているかを、3つの形で確かめた。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.22 rady = 0.22 radz = 0.22 rows = 24 ty = -0.9 type = 2
src	pyrosource	attribute1 = 0 attribute2 = 1 attributes = 2
rasterize	volumerasterizeattributes	attributes = density temperature folder1 = 1 folder3 = 1 voxelsize = 0.06

ノード	タイプ	デフォルトから変更したパラメータ
solve	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation = 0.05 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 …ほか 100 件
smoke_cache	filecache::2.0	f1 = 1.0 f2 = 30.0 file = D:/Claude/houdini/out/027_cache/smoke.0001.bgeo.sc filemethod = 1 folder2 = 1 folder3 = 1 loadfromdisk = 1

1. キャッシュを通して値は変わらないか

「ほぼ同じ」ではなく「完全に同じ」を求める。ここが崩れると、以降の測定がすべて信用できなくなる。

フレーム	計算した値	読んだ値	差
1	4,230.662863	4,230.662863	0.0000000000
10	30,202.378274	30,202.378274	0.0000000000
20	37,789.693870	37,789.693870	0.0000000000
30	43,739.861589	43,739.861589	0.0000000000

2. 速さと、2回目の振る舞い

2回目は同じ設定で呼び直した。指紋が一致するので、書き出しは起きないはずである。

やったこと	書き出し	読み出し	結果
計算しながら30フレーム回す	—	0.83 秒	これが基準
1回目の cache_sim()	0.12 秒	0.15 秒	計算より 5.5倍速い
2回目の cache_sim() (同じ設定)	起きない	0.20 秒	1回目と完全に一致

3. 上流を1つ変えたら、気づくか

pyrosolver の dissipation を、既定の 0.1 から 0.05 に下げた。それ以外は何も触っていない。



Houdini

dissipation 0.1 (既定) 。 F30。



Houdini

dissipation
0.05。同じカメラ・同じフレーム。頭が濃く大きい。

調べたこと	結果
指紋	8f35eab0ca26… → 366303d3368d… (変わった)
cache_sim() の判断	fingerprint changed → 計算し直した
値が変わったフレーム	29 / 30
F30 の密度の合計	43,739.86 → 66,890.32 (+52.9%)
指紋のファイルを壊した場合	no fingerprint → 計算し直した

わかったこと

- ・ キャッシュを通しても値は1ビットも変わらなかった。30フレームすべてで差が 0.0000000000。025の結論が、道具に組み込んだあとでも保たれている。
- ・ 2回目は書き出しが起きなかった。指紋が一致したので読むだけになり、値も1回目と完全に一致した。使う側は毎回 cache_sim() を呼ぶだけでよい。
- ・ 上流を1つ動かしたら、指紋が変わって計算し直した。これが無ければ、30フレーム中29フレームで違う値を、正しい値として読んでいたことになる。F30 で +52.9% のずれ。
- ・ 指紋は上流のノードをたどって、種類と「既定から動かしたパラメータ」だけを集めて作る。既定値を含めないで、Houdiniの版が上がって既定が増えても指紋は暴れない。
- ・ 指紋のファイルが壊れていたら、読まずに計算し直す。壊れたJSONを置いて確かめた。「読めないときは安全側に倒す」ー迷ったら計算し直すほうが、静かに間違えるよりよい。
- ・ 同じ計算を2回書き出すと、ファイルの合計サイズが 138バイト違った (38,927,958 → 38,928,096) 。中身の値は同じでも、ファイルはバイト単位では一致しない。つまりキャッシュの正しさをファイルの比較で確かめることはできない。指紋を「ファイル」ではなく「ネットワーク」から作っている理由がこれ。
- ・ 書き出したファイルの数がフレーム数と違えば、cache_sim() は例外を投げて止まる。実験025で40フレームのつもりが 240フレーム書かれて 3.5GB を無駄にした件を、道具の側で塞いだ。
- ・ pyrosolver の dissipation は既定で有効・値は 0.1。何も触らなくても煙は薄くなり続けている。半分にしかたけで F30 の密度が +52.9% になる。「既定=何もしていない」ではない。

- ・ hython は、レンダしたあとにシーンを作り直すと固まる。今回この順で10分待っても進まなかった。レンダは1プロセスの最後に1枚だけにして、比較の2枚目は別プロセスで撮る。カメラをそろえるため、枠を JSON に残して次のプロセスが読む。

次にやること

- ・ 重い実験 (Karma、FLIP、大きな Pyro) を `cache_sim()` に載せ替える
- ・ 028 として、まだ触っていない分野に進む
- ・ 実験030に達したら、001~030の振り返り点検を行う

升目を半分にしても、数は8倍ではなく約4倍 — VDB は表面の近くだけ持っている

2026-09-24

何を作ったか

VDB は空間を升目に区切って値を入れる。1辺を半分にすれば、空間ぜんぶを持つなら升目の数は8倍になる。ところが VDB

は表面の近くだけを持つ作りなので、面積に比例して4倍で済むのではないかな。測った。

球（50×50）に mountain をかけた形を、升目 0.2 から 0.0125 まで半分ずつ細かくして、VDB が実際に持っている升目の数（activevoxelcount）を数えた。

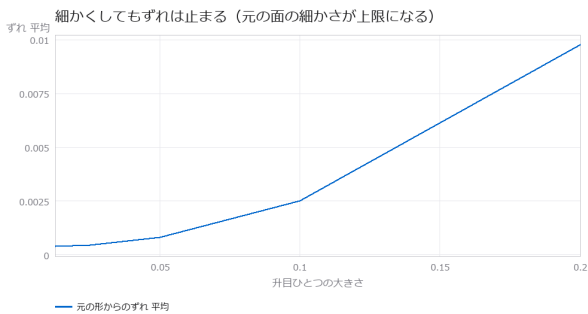
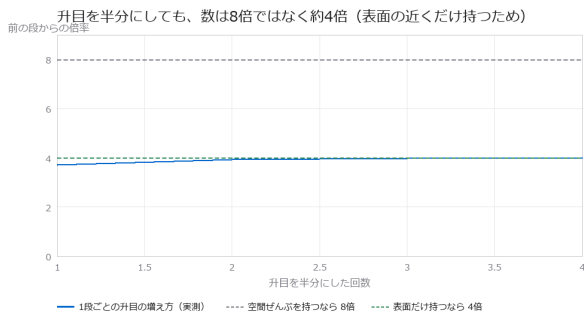
升目の数は 2098 → 483315。前の段からの倍率は 3.706 / 3.917 / 3.975 / 3.993 で、細かくするほど4に近づいた。**8倍にはならない。**

面に戻したときの面の数も同じように増える。446 面から 120606 面。

ずれは頭打ちになる。0.009767 から 0.000447 までは順に小さくなるが、最後の段では 0.000406 で、ほとんど変わらなかった。升目を細かくしても、元の面の細かさより細かい形は出てこない。

升目の大きさごとの、升目の数・面の数・ずれ

元は球（50×50）に mountain（高さ0.15）をかけた 2,450面。升目の数は、VDB の prim に activevoxelcount を聞いた値。「作る」は vdbfrompolygons、「戻す」は convertvdb の時間。



1段ごとの増え方は4に近づく。空間ぜんぶではなく、表面の近くだけを持っている。

細かくしてもずれは止まる。元の面の細かさが上限になる。

升目	升目の数	倍率	戻した面	作る秒	戻す秒	ずれ 平均
0.2	2,098	—	446	0.021	0.004	0.009767
0.1	7,775	3.706	1,876	0.010	0.001	0.002521
0.05	30,451	3.917	7,534	0.010	0.002	0.000804
0.025	121,050	3.975	30,134	0.013	0.005	0.000447
0.0125	483,315	3.993	120,606	0.022	0.013	0.000406

わかったこと

・ 8倍ではなく約4倍。実測 3.706 / 3.917 / 3.975 / 3.993。VDB は表面の近くだけを持つので、空間の体積ではなく面積に比例して増える。

- ・ずれは頭打ちになる。0.000447 から 0.000406 へ、最後の段では1.1倍しか縮まなかった。元の面の細かさが上限になっている。
- ・時間は升目の数ほど増えない。作るのは 0.0214秒 から 0.0219秒。升目が230倍になっても、この大きさでは1秒を切ったままだった。

次にやること

- ・元の面を細かくすれば、ずれはさらに小さくなるのか
- ・升目を細かくしたときの記憶の使用量
- ・SDF と Fog で、持つ升目の数が違うのか

Pyro で焚き火の炎（高さ 1.2 m）を作るなら Flame Lifespan 0.25・Buoyancy 0.25・Cooling Rate 1 — 揺らすのは Turbulence で、Use Control Field を切る

2026-09-24

何を作ったか

**課題: 焚き火くらいの炎（高さ 1 m 前後で揺れる）にするには、Pyro のどのつまみを動かすか。 **

幅 0.6・厚み 0.18 の薪（平たい球）を pyrosource の Initialize = Source Burn
で燃やし、pyrosolver（SOP、Voxel Size 0.04）で 72 フレーム（3 秒）回した。炎（flame >
0.1）の上から 1% の点の高さを毎フレーム測り、フレーム 36~72
の平均を「落ち着いた高さ」とした。炎を正面に投影した画から、輪郭の入り組み具合（縁の長さ ÷
面積の平方根）と、フレーム 48 と 72 の形の重なり（IoU、低いほどよく動く）も出した。

**既定のままでは焚き火にならない。 **炎は 1 秒に約 6 m で伸び、3 秒たっても平均 13.8 m
の柱のまま。

高さは3つのつまみで決まる。1つずつ変えた結果: Flame Lifespan（既定 2 秒）を $1 \cdot 0.5 \cdot 0.25$
にすると $12.3 \cdot 9.8 \cdot 4.1$ m。Lifespan 0.5 のまま Buoyancy Scale を $0.5 \cdot 0.25$ にすると $6.4 \cdot 4.0$
m。Cooling Rate を $0.75 \cdot 1$ にすると $9.1 \cdot 3.82$ m で、 $1.5 \cdot 2$ も $3.82 \cdot 3.82$ m と 1 と同じ（1
以上は効かない）。

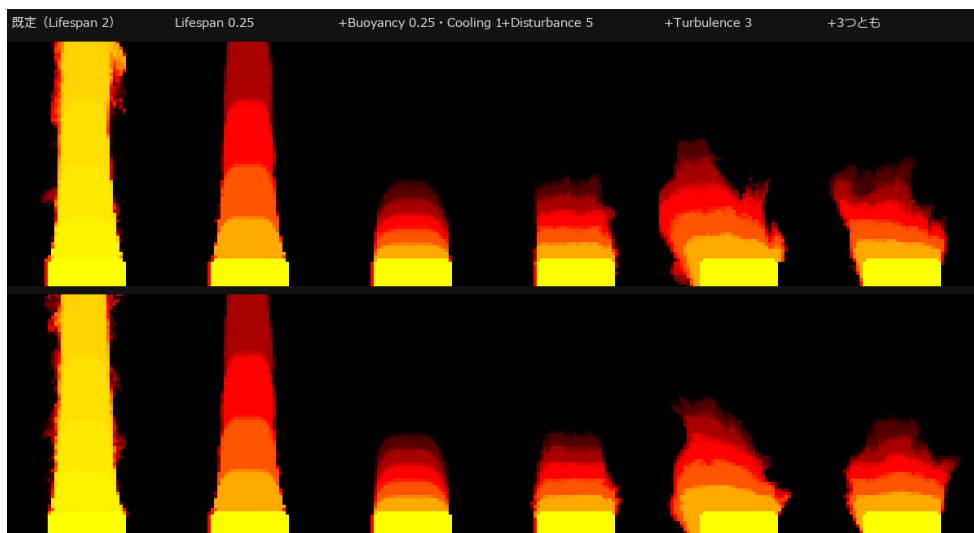
**組み合わせると 1 m 前後になる。 **Lifespan 0.25・Buoyancy 0.25・Cooling 1 で 1.21
m、Buoyancy を 0.1 にすると 0.90 m。

**ただしそのままでは炎が止まって見える。 **形の重なり（IoU）は 0.978（既定の柱でも
0.862）。揺らす仕組みを入れて比べた: Disturbance（既定の強さ・5）で $0.976 \cdot 0.925$ 、Shredding 2
で 0.939、Turbulence 1・3（Use Control Field を切る）で $0.805 \cdot 0.795$ 。輪郭の入り組み具合も
Turbulence 3 が 5.89 と一番大きい（揺らす前は 4.27）。Turbulence 3 を入れても高さは 1.37 m
にとどまった。

**Turbulence を入れても効かなかったのは、Use Control Field のせい。 **既定（Use Control Field
入、制御する場合は density）のまま Turbulence を 1 にしても、高さ 1.214 m・IoU 0.978
と、入れる前と小数 3 桁まで同じ。切ると 1.310 m・IoU 0.805 に変わった。

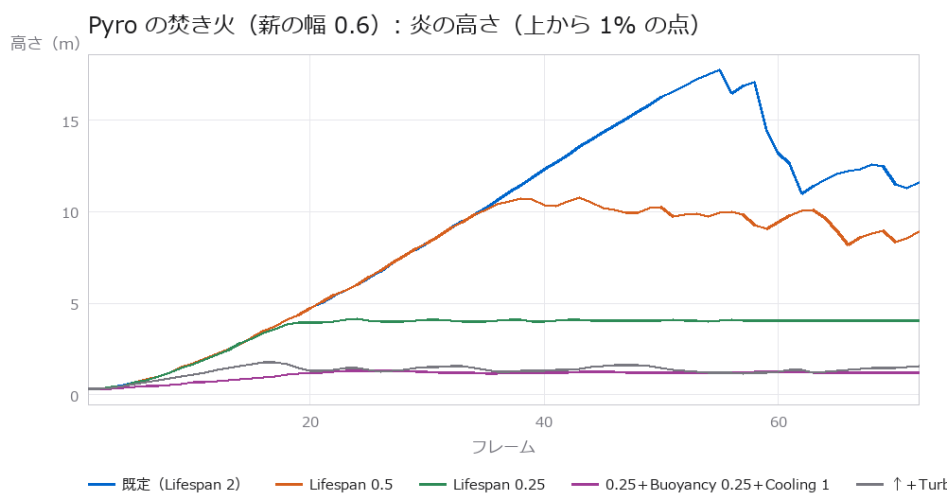
**時間は炎が小さいほど短い。 **既定 16.6 秒に対し、焚き火の設定は 2.5 秒、Turbulence 3 を足して
4.3 秒（72 フレーム）。

炎を正面に投影した画（上がフレーム48、下がフレーム72）



左から 既定・Lifespan 0.25・焚き火の設定・+Disturbance 5・+Turbulence 3・+3つとも。

条件と、炎の高さ・動き



既定は伸び続ける。Lifespan と Buoyancy・Cooling で 1 m 台に収まる。

条件	落ち着いた高さ (m)	高さの揺らぎ (m)	伸びる速さ (m /秒)	形の重なり F48-72	輪郭の入り組み	秒
life2	13.76	2.201	6.0	0.862	5.37	16.6
life1	12.29	1.894	6.0	0.884	5.46	14.7
life05	9.76	0.700	6.0	0.894	4.86	8.6
life025	4.06	0.020	6.0	0.982	5.04	3.8
buoy05	6.40	0.151	3.6	0.937	5.09	5.0
buoy025	4.00	0.115	2.0	0.948	4.82	3.8
cool1	3.82	0.037	3.2	0.919	3.77	4.9
cool2	3.82	0.037	3.2	0.919	3.77	4.9
cool075	9.10	0.723	6.0	0.933	5.05	6.8
cool15	3.82	0.037	3.2	0.919	3.77	4.2

条件	落ち着いた高さ (m)	高さの揺らぎ (m)	伸びる速さ (m/秒)	形の重なり F48-72	輪郭の入り組み	秒
l025b025	1.75	0.062	1.9	0.974	4.73	2.4
l025c1	2.06	0.027	2.9	0.968	4.53	2.9
l025b025c1	1.21	0.033	1.1	0.978	4.27	2.5
l025b01c1	0.90	0.029	0.6	0.976	4.19	2.3
l025b025c1_dist	1.22	0.032	1.1	0.976	4.30	2.8
l025b025c1_turb	1.21	0.033	1.1	0.978	4.27	2.7
fire_dist2	1.23	0.025	1.1	0.950	4.56	2.5
fire_dist5	1.21	0.030	1.2	0.925	4.88	2.5
fire_turb1	1.31	0.075	2.1	0.805	5.33	3.1
fire_turb1_ctrl	1.21	0.033	1.1	0.978	4.27	2.4
fire_turb3	1.37	0.136	2.7	0.795	5.89	4.3
fire_shred2	1.22	0.027	1.1	0.939	4.52	2.9
fire_mix	1.30	0.065	2.1	0.789	5.34	4.1

わかったこと

- ・ 焚き火の高さ（1 m 台）は Flame Lifespan 0.25 ・ Buoyancy Scale 0.25 ・ Cooling Rate 1。既定の Lifespan 2 秒では、炎が柱になって伸び続ける。
- ・ Cooling Rate は 1 より上げて変わらない。
- ・ 炎を揺らすのは Turbulence（1～3）。Disturbance と Shredding は輪郭を少し乱すだけで、炎全体はあまり動かない。
- ・ Turbulence が効かないときは Use Control Field を切る。既定のまま（制御する場 density）だと、強さを上げて何も変わらなかった。

焚き火の Pyro を速く回すなら Voxel Size を 0.02 → 0.04 に — 5.1 倍速く、炎の高さの差は 3%。速度の升だけ粗くするのと Substeps 2 は、かえって遅い

2026-09-24

何を作ったか

**課題: Pyro の焚き火 (高さ 1.5 m 前後、実験201 の設定) を短い時間で回したい。どのつまみを下げればよく、見た目はどれだけ変わるか。 **

平たい球 (半径 0.3 ・ 厚み 0.18) を pyrosource の Source Burn で燃やし、 pyrosolver (SOP) に実験201 の焚き火の設定 (Flame Lifespan 0.25 ・ Buoyancy 0.25 ・ Cooling 1 ・ Turbulence 3 ・ Use Control Field 切) を入れて 48 フレーム (2 秒) 回した。 Voxel Size を 0.02 (基準) ・ 0.03 ・ 0.04 ・ 0.06 に変えたもの、0.02 のまま Velocity Voxel Scale を 2 にしたもの、Max Substeps を 2 にしたものを、1 プロセスずつ測った。毎フレーム、炎 (flame > 0.1) の上から 1% の点の高さを出し、フレーム 24~48 の平均を「炎の高さ」とした。フレーム 24 ・ 36 ・ 48 の炎を正面に投影し、基準との重なり (IoU) も出した。

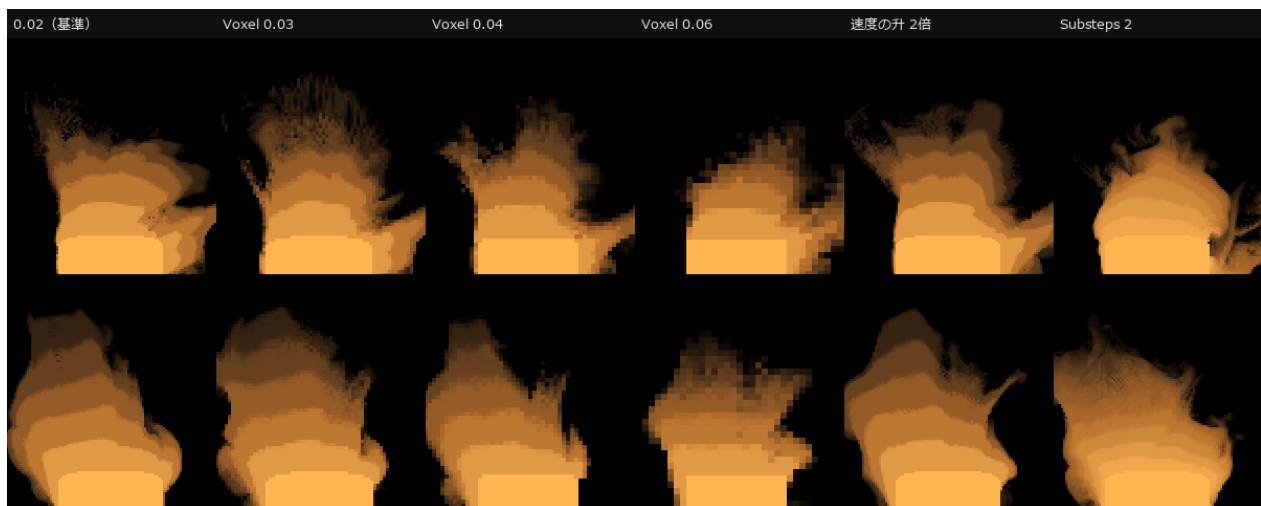
**Voxel Size がいちばん効く。 **0.02 の 26.0 秒に対し、0.03 で 9.1 秒 (2.8 倍速い) 、0.04 で 5.1 秒 (5.1 倍) 、0.06 で 2.3 秒 (11.4 倍) 。炎の高さは 1.47 ・ 1.48 ・ 1.42 ・ 1.35 m で、0.04 までは 3% 以内、0.06 で 8% 低くなった。

**速度の升だけ粗くする (Velocity Voxel Scale 2) は速くならなかった。 **33.9 秒で、基準より遅い。最後のフレームの升の数は [160, 192, 192] (基準は [128, 160, 176]) と、計算する範囲のほうが広がっていた。

**Max Substeps 2 も遅い (32.8 秒) 。 **高さは 1.41 m で、揺らぎ (高さの標準偏差) は 0.164 m と、基準の 0.127 m より大きい。速くするための設定ではない。

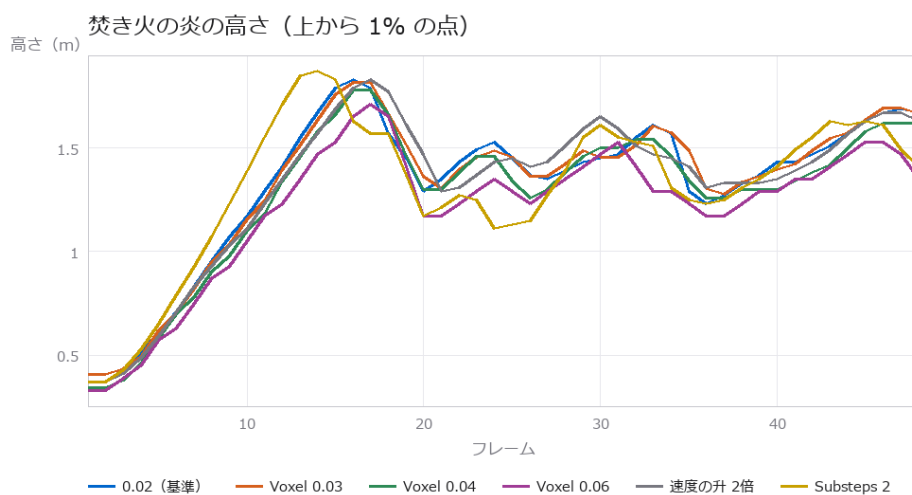
**形の重なり (IoU) は、どの条件も 0.72~0.89。 **高さがほぼ同じ Voxel 0.03 でも 0.75~0.89、速度の升 2倍でも 0.77~0.87。炎の細かい形はどの変更でもずれるので、形を1画素ずつ合わせる比べ方では差が出すぎる。ここでは高さで判断した。

炎と煙を正面に投影した画 (上がフレーム36、下がフレーム48)



左から Voxel 0.02 ・ 0.03 ・ 0.04 ・ 0.06 ・ 速度の升 2倍 ・ Substeps 2。炎は橙、煙は灰色。

条件ごとの時間と炎



炎の高さの移り変わり。0.06 だけが目に見えて低い。

条件	48 フレームの計算 (秒)	基準より何倍速い	炎の高さ (m)	高さの揺らぎ (m)	形の重なり (IoU)
0.02 (基準)	26.0	1.0	1.47	0.127	1.00~1.00
Voxel 0.03	9.1	2.8	1.48	0.116	0.75~0.89
Voxel 0.04	5.1	5.1	1.42	0.119	0.74~0.86
Voxel 0.06	2.3	11.4	1.35	0.106	0.71~0.77
速度の升 2倍	33.9	0.8	1.48	0.113	0.77~0.87
Substeps 2	32.8	0.8	1.41	0.164	0.72~0.80

わかったこと

- ・ 焚き火を速く回すなら Voxel Size を上げる。0.02 → 0.04 で 5.1 倍速く、炎の高さの差は 3% 以内。形を決めるうちは 0.04、仕上げで 0.02 に戻す。
- ・ 0.06 まで上げると炎が 8% 低くなる。速さは 11 倍だが、高さを合わせたい段階では使わない。

- ・ Velocity Voxel Scale 2 と Max Substeps 2

は、この焚き火では遅くなった。速くする目的で触るつまみではない。

- ・ 炎の細かい形は、どの設定を変えてもずれる（重なり

0.72～0.89）。速さの設定を選ぶときは、形の重なりより高さを見る。